

Şirketlerin Dijital Çalışanı

AI Destekli Kurumsal Bilgi Asistanı — MVP Proje Planı

Bootcamp Proje Ekibi

Temmuz 2026

Contents

1. Projenin Amacı	2
Hangi problemi çözüyor?	2
Hedef kullanıcı kim?	2
Kullanıcı bu sistemi neden kullanır?	3
MVP'de çözmemiz gereken en temel problem	3
2. MVP Kapsamı	3
MVP'de kesinlikle olması gereken özellikler	3
MVP dışında bırakılması gereken özellikler	3
İlk versiyonda fazla karmaşıklıştırmadan nasıl geliştirilebilir?	3
Bootcamp süresine uygun, gerçekçi bir kapsam önerisi	4
3. Kullanıcı Akışı / İş Akışı	4
Dokümanlar Nereden ve Nasıl Toplanmalı?	4
Hangi doküman türleri öncelikli?	4
Dokümanlar nereden temin edilir? (bootcamp / demo bağlamında)	4
Dosya türü ve format kuralları	4
Organizasyon önerisi (kategori/etiketleme)	5
Kullanıcı sisteme girdiğinde adım adım ne yapar?	5
Backend tarafında bu istekler nasıl işlenir?	5
AI hangi aşamada devreye girer?	5
Veriler nerede saklanır?	6
Sonuç kullanıcıya nasıl döner?	6
4. Teknik Mimari	6
Monolith mi, mikroservis mi?	6
Servislerin ayrımı	6
Servisler birbirleriyle nasıl iletişim kurar?	6
API Gateway gerekli mi?	7
5. Teknoloji Araştırması ve Stack Önerisi	7
Frontend	7
Backend	7
AI / ML / LLM tarafı	7
Veritabanı	7
Vektör veritabanı	8
Authentication	8
Docker / Docker Compose	8
Deployment seçenekleri	8
Logging ve error handling	8
API dokümantasyonu	8

6. Mikroservis Klasör ve Dosya Yapısı	8
Klasörlerin açıklaması	11
7. API Contract / Endpoint Planı	11
Auth Service	11
Document Service	12
Ingestion Service (genelde iç servis, doğrudan frontend'e açık değil)	12
Chat Service	12
Ortak Response Formatı	13
Hata Response Formatı	13
Servisler arası veri alışverişi	13
8. Geliştirme Yol Haritası	14
Aşama 1 — Proje Setup (2-3 gün)	14
Aşama 2 — Shared Modeller ve Ortak Altyapı (2 gün)	14
Aşama 3 — Backend Servisleri (5-6 gün)	14
Aşama 4 — AI Entegrasyonu (4-5 gün)	14
Aşama 5 — Frontend (5-6 gün, Aşama 3-4 ile paralel yürütülebilir)	14
Aşama 6 — Docker Compose Entegrasyonu (2-3 gün)	15
Aşama 7 — Test ve Demo Hazırlığı (3-4 gün)	15
9. Karşılaşılabilecek Problemler ve Çözüm Önerileri	15
10. İlgili Çekici Ek Özellikler (MVP Sonrası, Zaman Kalırsa)	16
Güven Skoru / Belirsizlik Göstergesi	16
"Bu Cevap Yardımcı Oldu mu?" Geri Bildirimi	16
Haftalık Özet / Sindirilebilir Bildirim	16
Doküman Güncellik Uyarısı	16
Slack/Teams Bot Entegrasyonu (Basit Versiyon)	17
Neden Dashboard'ı Şimdilik Kapsam Dışı Bıraktık?	17
Özet	17

1. Projenin Amacı

Hangi problemi çözüyor?

KOBİ'lerde kurumsal bilgi; e-postalarda, paylaşılan dosyalarda, Excel tablolarında, WhatsApp yazışmalarında ve insanların kafasında dağınık halde durur. Bunun sonucunda:

- Çalışanlar aynı soruları tekrar tekrar birbirlerine sorar ("Bu müşteriye geçen sefer nasıl teklif verdik?").
- Yeni işe başlayan biri günlerce/haftalarca "iş öğrenmeye" harcar.
- Kritik bilgi (sözleşme, prosedür, geçmiş karar) kaybolur ya da yanlış versiyon kullanılır.
- Bilgiyi bilen kişi izinliyse ya da işten ayrılmışsa, o bilgi de gitmiş olur.

Hedef kullanıcı kim?

- **Birincil hedef:** 10-100 çalışanlı KOBİ'ler (danışmanlık, ajans, üretim, hizmet sektörü).
- **Kullanıcı personaları:**
 - **Operasyon çalışanı:** Günlük işinde sürekli bilgiye ihtiyaç duyan kişi (satış temsilcisi, müşteri temsilcisi, proje yöneticisi).
 - **Yeni işe başlayan:** Onboarding sürecinde şirket bilgisine hızlı erişmesi gereken kişi.
 - **Yönetici/Departman sorumlusu:** Şirket dokümanlarını sisteme yükleyen, erişim yetkilerini yöneten kişi (admin rolü).

Kullanıcı bu sistemi neden kullanır?

- Doğal dilde soru sorup saniyeler içinde, kaynağı belli olan bir cevap alır.
- Aynı soruyu bir insana sormak yerine sisteme sorar, meslektaşının zamanını almaz.
- Cevabın hangi dokümandan/hangi tarihte geldiğini görerek güvenilirliğini kontrol edebilir.

MVP’de çözmemiz gereken en temel problem

“Şirket içindeki günlük dokümanları tek bir yerden, doğal dilde soru sorarak, kaynak göstererek sorgulayabilme.”

Bu tek cümle MVP’nin kalbidir. E-posta entegrasyonu, otomatik prosedür güncelleme, öğrenen sistem gibi özellikler MVP sonrasına bırakılır.

2. MVP Kapsamı

MVP’de kesinlikle olması gereken özellikler

1. **Kullanıcı kayıt/giriş** (şirket bazlı, basit JWT authentication).
2. **Doküman yükleme** — PDF, DOCX, TXT formatlarını destekleyen upload arayüzü.
3. **Doküman işleme (ingestion)** — yüklenen dokümanın parçalanıp (chunking) vektör veritabanına kaydedilmesi.
4. **Soru-cevap arayüzü** — kullanıcının doğal dilde soru sorabildiği bir chat ekranı.
5. **RAG (Retrieval-Augmented Generation) pipeline** — soruya en alakalı doküman parçalarının bulunup LLM’e context olarak verilmesi.
6. **Kaynak gösterimi** — cevabın hangi dokümandan/hangi bölümden geldiğinin gösterilmesi.
7. **Doküman listesi/yönetimi** — yüklenen dokümanları görüntüleme, silme.
8. **Basit sohbet geçmişi** — kullanıcının önceki sorularını görebilmesi.

MVP dışında bırakılması gereken özellikler

- E-posta / Outlook / Gmail entegrasyonu
- Slack / Teams entegrasyonu
- Otomatik prosedür güncelleme ve versiyon takibi
- Çoklu dil desteği (sadece Türkçe/İngilizce ile başla)
- Detaylı rol bazlı yetkilendirme (departman bazlı erişim kısıtlama)
- Kullanım analitiği / dashboard (kim ne sordu, en çok sorulan konular)
- Mobil uygulama
- Gerçek zamanlı doküman senkronizasyonu (Google Drive, OneDrive bağlantısı)
- Fine-tuning / şirkete özel model eğitimi
- SaaS çoklu tenant altyapısı (ödeme sistemi, plan yönetimi)

İlk versiyonda fazla karmaşıklıştırmadan nasıl geliştirilebilir?

- Tek şirket / tek workspace mantığıyla başla, “multi-tenant” mimariyi MVP sonrasına bırak ama veri modelini ileride tenant eklemeye izin verecek şekilde kur (her tabloda company_id alanı bulunsun).
- Kullanıcı rolü olarak sadece iki rol yeterli: admin (doküman yükler) ve employee (sadece soru sorar).
- Dosya formatı desteğini PDF + DOCX + TXT ile sınırla, Excel/PowerPoint desteğini sonraya bırak.
- LLM olarak kendi model eğitmek yerine hazır bir API (OpenAI, Anthropic, veya açık kaynak bir model) kullan.

Bootcamp süresine uygun, gerçekçi bir kapsam önerisi

4-6 haftalık bir bootcamp için gerçekçi versiyon:

- **Hafta 1:** Proje setup, mimari tasarım, veritabanı şeması, temel auth.
- **Hafta 2:** Doküman upload + ingestion pipeline (chunking, embedding, vektör DB'ye kayıt).
- **Hafta 3:** RAG pipeline + soru-cevap servisinin çalışır hale gelmesi.
- **Hafta 4:** Frontend geliştirme, chat arayüzü, doküman yönetim ekranı.
- **Hafta 5:** Docker Compose ile entegrasyon, servisler arası iletişimin test edilmesi, hata düzeltme.
- **Hafta 6:** Test, demo senaryosu hazırlama, sunum, dokümantasyon.

3. Kullanıcı Akışı / İş Akışı

Dokümanlar Nereden ve Nasıl Toplanmalı?

MVP'de kaynak tek nokta: **manuel upload**. Ama “manuel upload” tek başına yeterli bir tanım değil — hangi dokümanların, kimden, nasıl toplanacağı netleşmezse admin kullanıcı ekrana geldiğinde ne yükleyeceğini bilemez. Bunun için pratik bir çerçeve:

Hangi doküman türleri öncelikli?

Bir KOBİ'de en çok “aynı soru tekrar tekrar sorulan” doküman türleri genelde şunlardır — MVP demo senaryosu da bunlardan seçilmeli:

1. **Prosedür/politika dokümanları** — izin prosedürü, masraf beyanı süreci, işe alım süreci vb. (genelde Word/PDF, az sayıda, yüksek etki).
2. **Sözleşme ve teklif şablonları/geçmişi** — müşteri teklifleri, hizmet sözleşmeleri (PDF ağırlıklı).
3. **SSS / onboarding dokümanları** — yeni çalışana verilen “hoş geldin” rehberleri.
4. **Toplantı notları / karar kayıtları** — “geçen yıl bu konu nasıl çözülmüştü” sorusuna cevap veren metinler (genelde Word/TXT, düzensiz olabilir).

MVP demo için 4 kategoriden en az 1-2'şer örnek doküman toplamak, “gerçek bir şirket verisiyle çalışıyor” izlenimi verir ve jüri karşısında inandırıcılığı artırır.

Dokümanlar nereden temin edilir? (bootcamp / demo bağlamında)

Gerçek bir şirket verisi olmadığı için demo verisi şu yollarla oluşturulabilir:

- **Sentetik ama gerçekçi dokümanlar üretmek** — LLM'e (“bir danışmanlık şirketinin izin prosedürü dokümanı yaz” gibi) prompt vererek 5-10 sayfalık gerçekçi örnek dokümanlar üretilir. En hızlı ve pratik yöntem budur.
- **Halka açık şablon dokümanlar** — internet üzerinde bulunan örnek sözleşme şablonları, prosedür şablonları (telif/gizlilik sorunu olmayan, “template” olarak paylaşılan dokümanlar).
- **Ekip üyelerinin kendi organizasyonlarından (üniversite kulübü, staj yaptıkları yer vb.) anonimleştirerek aldığı, hassas bilgi içermeyen örnek dokümanlar.**
- **Gerçek bir pilot şirket bulunabilirse** (bootcamp mentörü/hocası aracılığıyla küçük bir işletmeyle görüşülebilir) — bu en güçlü demo senaryosunu yaratır ama zaman riski taşır, MVP teslim tarihine yakın denenmemeli.

Dosya türü ve format kuralları

Format	MVP’de destek	Not
PDF (metin tabanlı)	[Var] Destekleniyor	En yaygın format, öncelik burada
DOCX	[Var] Destekleniyor	Word şablonları için gerekli
TXT	[Var] Destekleniyor	Toplantı notları gibi basit metinler için
Taranmış/görüntü PDF (OCR gerektiren)	[Yok] MVP dışı	Kapsam dışı bırakılmalı, yüklenirse net hata mesajı verilmeli
Excel/PowerPoint	[Yok] MVP dışı	Tablo/görsel içerik RAG için ayrı bir işleme stratejisi gerektirir
Boyut sınırı	Öneri: dosya başına max 10-15 MB	Büyük dosyalarda ingestion süresi ve maliyet artar

Organizasyon önerisi (kategori/etiketleme)

Upload ekranında dokümanı yüklerken admin’e basit bir **kategori seçimi** yaptırmak (örn. “Prosedür”, “Sözleşme”, “Toplantı Notu”, “Diğer”) ileride: - Cevap kaynağını daha anlamlı göstermeyi sağlar (“Bu bilgi bir *prosedür* dokümanından geliyor” gibi). - Gelecekte kategori bazlı filtreleme/arama eklemeyi kolaylaştırır.

Bu, MVP’yi karmaşıktırmadan eklenebilecek küçük ama etkili bir detaydır — veritabanı şemasında documents tablosuna bir category alanı eklemek yeterlidir.

Kullanıcı sisteme girdiğinde adım adım ne yapar?

1. Kullanıcı giriş yapar (email + şifre).
2. **Admin ise:** “Dokümanlar” sayfasına gidip PDF/DOCX/TXT dosyası yükler.
3. **Employee ise:** Doğrudan “Sohbet” ekranına gelir ve doğal dilde sorusunu yazar: “*Bu müşteriye daha önce nasıl teklif vermiştik?*”
4. Sistem cevabı üretir ve ekrana; (a) özet cevabı, (b) kaynak doküman adı/bölümünü, (c) “kaynağı görüntüle” linkini gösterir.
5. Kullanıcı isterse aynı sohbet penceresinde takip sorusu sorabilir.
6. Kullanıcı geçmiş sohbetlerine “Sohbet Geçmişi” bölümünden ulaşabilir.

Backend tarafında bu istekler nasıl işlenir?

Doküman yükleme akışı: 1. Frontend, dosyayı Document Service’e gönderir. 2. Document Service dosyayı obje depolamaya (örn. local disk veya S3-uyumlu storage) kaydeder, metadata’yı veritabanına yazar. 3. Document Service, işlem sırasını Ingestion Service’e iletir (mesaj kuyruğu veya direkt API çağrısı ile). 4. Ingestion Service dosyayı okur, metne çevirir (PDF/DOCX parser), parçalara böler (chunking), her parça için embedding vektörü üretir ve vektör veritabanına kaydeder.

Soru-cevap akışı: 1. Frontend, kullanıcının sorusunu Chat Service’e gönderir. 2. Chat Service, soruyu embedding’e çevirir, vektör veritabanında en alakalı doküman parçalarını arar (similarity search). 3. Bulunan parçalar + kullanıcının sorusu, AI Service üzerinden LLM’e prompt olarak gönderilir. 4. LLM, sadece verilen context’e dayanarak cevap üretir (hayal görmeyi/uydurmaya azaltmak için “sadece verilen bilgiye dayan” talimatı verilir). 5. Chat Service, cevabı + kaynak referanslarını birlikte veritabanına (sohbet geçmişi) kaydeder ve frontend’e döner.

AI hangi aşamada devreye girer?

İki noktada: 1. **Embedding üretimi** — hem doküman parçaları hem kullanıcı sorusu için (ingestion ve retrieval sırasında). 2. **Cevap üretimi (generation)** — bulunan context’e dayanarak doğal dilde özet cevap oluşturmak için (LLM çağrısı).

Veriler nerede saklanır?

Veri türü	Saklama yeri
Kullanıcı, şirket, doküman metadata, sohbet geçmişi	PostgreSQL (ilişkisel veritabanı)
Ham dosyalar (PDF, DOCX)	Local disk / MinIO (S3 uyumlu object storage)
Doküman parçaları + embedding vektörleri	Vektör veritabanı (örn. Qdrant, ChromaDB, pgvector)
Oturum / cache	Redis (opsiyonel, MVP’de basit tutulabilir)

Sonuç kullanıcıya nasıl döner?

Frontend’de chat balonu şeklinde; cevap metni + altında küçük bir “Kaynaklar” kutusu (doküman adı, sayfa/bölüm numarası, “görüntüle” linki) gösterilir. Standart bir JSON response formatı ile (bkz. Bölüm 7) frontend’e iletilir.

4. Teknik Mimari

Monolith mi, mikroservis mi?

Öneri: Hafif mikroservis mimarisi (3-4 servis), ama abartısız.

Gerekçe: Bootcamp’in amacı hem çalışan bir ürün çıkarmak hem de mikroservis mimarisi deneyimi kazanmaktır. Tam bir “enterprise” mikroservis mimarisine (10+ servis, service mesh, event sourcing vb.) gerek yok — bu bootcamp süresine sığmaz ve gereksiz karmaşıklık yaratır. Bunun yerine, **sorumluluğu net ayrılmış, az sayıda servis** tercih edilmeli.

Eğer ekip küçükse (2-3 kişi) ve süre kısıtlıysa, **modüler monolith** de meşru bir alternatiftir; ama bootcamp’te mikroservis deneyimi hedefleniyorsa aşağıdaki 4 servisli yapı önerilir.

Servislerin ayrımı

Servis	Sorumluluk
Auth Service	Kullanıcı kayıt/giriş, JWT üretimi/doğrulama, şirket (company) yönetimi
Document Service	Dosya upload, metadata yönetimi, dosya storage ile iletişim, doküman silme/listeleme
Ingestion Service	Dosyayı metne çevirme, chunking, embedding üretimi, vektör DB’ye kayıt
Chat/AI Service	Kullanıcı sorusunu alma, vektör arama, LLM’e prompt gönderme, cevabı formatlayıp döndürme, sohbet geçmişi kaydı

Frontend bu 4 servisle API Gateway üzerinden konuşur.

Servisler birbirleriyle nasıl iletişim kurar?

- **MVP için:** Senkron HTTP/REST (basit ve debug’ı kolay). Örneğin Document Service, dosya kaydedildikten sonra Ingestion Service’e bir HTTP POST isteği atar (/process-document).

- **İleri seviye (opsiyonel, zaman kalırsa):** Document Service ile Ingestion Service arasına bir mesaj kuyruğu (RabbitMQ/Redis Queue) koymak, böylece büyük dosyalarda upload isteği hemen dönebilir, işleme arka planda devam eder. Bootcamp için bu **opsiyonel bir “nice-to-have”** olarak düşünölmeli, MVP’yi bloklamamalı.
- Servisler arası kimlik doğrulama: Auth Service’in ürettiğı JWT, diğör servislere Authorization header’ı ile taşınır; her servis JWT’yi doğrulayan ortak bir middleware kullanır (bkz. shared klasörü).

API Gateway gerekli mi?

Evet, basit bir API Gateway önerilir, ama karmaşık bir Kong/Istio kurulumu değil. Nedenleri:

- Frontend’in 4 farklı servisin portunu/adresini bilmesine gerek kalmaz, tek bir adrese (api.projemiz.com) istek atar.
- CORS, rate limiting, ortak authentication kontrolü tek noktadan yapılabilir.
- İleride yeni servis eklemek/route değiştirmek frontend’i etkilemez.

Bootcamp için **basit bir reverse proxy (Nginx) veya hafif bir Node.js/FastAPI gateway** yeterlidir; tam bir API Gateway ürünü (Kong, Apigee) gereksizdir.

5. Teknoloji Araştırması ve Stack Önerisi

Frontend

- **React + TypeScript** — bootcamp’lerde en yaygın, geniş topluluk desteğı.
- **Vite** — hızlı geliştirme ortamı için (Create React App yerine).
- **TailwindCSS** — hızlı ve tutarlı arayüz geliştirme.
- **Axios veya React Query** — API çağrıları için.
- Alternatif: Next.js (eğer SSR/SaaS’a dönüştürme hedefi öne çıkarsa daha uygun olabilir, ama MVP için React+Vite yeterli ve daha basit).

Backend

- **Python + FastAPI** — hem hızlı geliştirme hem de AI/ML kütüphaneleriyle (LangChain, embedding modelleri) doğal uyum sağlar. Otomatik API dokümantasyonu (Swagger) ücretsiz gelir.
- Alternatif: Node.js + Express/NestJS (ekip JS’e daha yatkınsa), ama AI tarafındaki Python kütüphane zenginliğı FastAPI’yi öne çıkarır.

AI / ML / LLM tarafı

- **LLM API:** Anthropic Claude API veya OpenAI API (hazır, kaliteli, hızlı entegre edilir; kendi model eğitmeye gerek yok).
- **RAG orkestrasyon:** LangChain veya LlamaIndex — chunking, retrieval, prompt şablonlama işlerini kolaylaştırır. Bootcamp’te “sıfırdan yazmak” yerine bu kütüphaneleri kullanmak zaman kazandırır.
- **Embedding modeli:** OpenAI text-embedding-3-small veya açık kaynak sentence-transformers (örn. all-MiniLM-L6-v2) — maliyet düşürmek istenirse açık kaynak tercih edilebilir.

Veritabanı

- **PostgreSQL** — kullanıcı, şirket, doküman metadata, sohbet geçmişi için ilişkisel veri.

Vektör veritabanı

- **Öneri: pgvector (PostgreSQL eklentisi)** — MVP için en pratik seçim. Ayrı bir vektör DB kurmaya gerek kalmaz, tek PostgreSQL instance'ı hem ilişkisel veriyi hem vektörleri tutar. Docker Compose'da tek bir servis azalır.
- Alternatif: **Qdrant** veya **ChromaDB** — eğer ileride ölçeklenme/performans öncelikliyse ayrı bir vektör DB'ye geçilebilir. Bootcamp süresi için pgvector yeterli ve basittir.

Authentication

- **JWT (JSON Web Token)** — stateless, mikroservis mimarisine uygun.
- Şifreleme: bcrypt ile şifre hashleme.
- Kütüphane: FastAPI için fastapi-users veya kendi basit JWT middleware'i.

Docker / Docker Compose

- Her mikroservis kendi Dockerfile'ına sahip olur.
- docker-compose.yml ile tüm servisler (auth, document, ingestion, chat, postgres, frontend, gateway) tek komutla ayağa kalkar: docker-compose up.

Deployment seçenekleri

- **Bootcamp/demo için:** Railway, Render, veya Fly.io — ücretsiz/düşük maliyetli, Docker Compose'a yakın deployment deneyimi sunar.
- **Daha “production-like” istenirse:** Bir VPS (DigitalOcean, Hetzner) üzerinde Docker Compose ile deploy.
- Kubernetes gibi bir orkestrasyon aracı **bootcamp kapsamı için gereksizdir**, öğrenme eğrisi zamanı çok tüketir.

Logging ve error handling

- Her serviste standart Python logging modülü, yapılandırılmış (JSON) log formatı.
- Ortak bir exception handler middleware'i (FastAPI'de @app.exception_handler) ile tutarlı hata response formatı (bkz. Bölüm 7).
- İleri seviye (opsiyonel): Sentry entegrasyonu ile hata takibi.

API dokümantasyonu

- FastAPI kullanılıyorsa **Swagger/OpenAPI otomatik gelir** (/docs endpoint'i) — ekstra efor gerektirmez, bootcamp jürisine gösterim için idealdir.

6. Mikroservis Klasör ve Dosya Yapısı

```
dijital-calisan/  
├── docker-compose.yml  
├── .env.example  
├── .gitignore  
├── README.md  
  
├── gateway/  
│   ├── Dockerfile  
│   ├── nginx.conf  
│   └── README.md
```



```

services/
├── auth-service/
│   ├── Dockerfile
│   ├── requirements.txt
│   ├── .env.example
│   └── app/
│       ├── main.py
│       ├── config.py
│       ├── database.py
│       ├── models/
│       │   └── user.py
│       ├── schemas/
│       │   └── auth_schema.py
│       ├── services/
│       │   └── auth_service.py
│       ├── routers/
│       │   └── auth_router.py
│       └── utils/
│           └── security.py
├── document-service/
│   ├── Dockerfile
│   ├── requirements.txt
│   ├── .env.example
│   └── app/
│       ├── main.py
│       ├── config.py
│       ├── database.py
│       ├── models/
│       │   └── document.py
│       ├── schemas/
│       │   └── document_schema.py
│       ├── services/
│       │   ├── storage_service.py
│       │   └── document_service.py
│       ├── routers/
│       │   └── document_router.py
│       └── storage/
│           └── (yüklenen dosyalar - local dev için)
├── ingestion-service/
│   ├── Dockerfile
│   ├── requirements.txt
│   ├── .env.example
│   └── app/
│       ├── main.py
│       ├── config.py
│       ├── database.py
│       ├── models/
│       │   └── chunk.py
│       ├── schemas/
│       │   └── ingestion_schema.py
│       ├── services/
│       │   └── parser_service.py
│       └──

```

PDF/DOCX -> text

```

├── chunking_service.py    # text -> chunks
├── embedding_service.py  # chunks -> vectors
├── routers/
│   └── ingestion_router.py
├── chat-service/
│   ├── Dockerfile
│   ├── requirements.txt
│   ├── .env.example
│   └── app/
│       ├── main.py
│       ├── config.py
│       ├── database.py
│       ├── models/
│       │   └── conversation.py
│       ├── schemas/
│       │   └── chat_schema.py
│       ├── services/
│       │   ├── retrieval_service.py    # vektör arama
│       │   ├── llm_service.py         # LLM API çağrısı
│       │   └── chat_service.py         # orkestrasyon
│       └── routers/
│           └── chat_router.py
├── shared/
│   ├── auth_middleware.py    # JWT doğrulama, tüm servislerde kullanılır
│   ├── response_format.py    # ortak response/hata formatı
│   ├── logger.py            # ortak logging config
│   └── constants.py          # ortak sabitler (roller, hata kodları vb.)
├── frontend/
│   ├── Dockerfile
│   ├── package.json
│   ├── .env.example
│   └── src/
│       ├── main.tsx
│       ├── App.tsx
│       ├── api/
│       │   ├── authApi.ts
│       │   ├── documentApi.ts
│       │   └── chatApi.ts
│       ├── pages/
│       │   ├── LoginPage.tsx
│       │   ├── DocumentsPage.tsx
│       │   └── ChatPage.tsx
│       ├── components/
│       │   ├── ChatBubble.tsx
│       │   ├── SourceCard.tsx
│       │   ├── DocumentUpload.tsx
│       │   └── DocumentList.tsx
│       └── hooks/
│           └── useAuth.ts

```

Klasörlerin açıklaması

- **gateway/** — Nginx tabanlı basit reverse proxy. Frontend'den gelen istekleri ilgili mikroservise yönlendirir (/auth/* → auth-service, /documents/* → document-service vb.).
- **services/** — Her mikroservisin kendi bağımsız klasörü. Her biri kendi Dockerfile, requirements.txt ve .env dosyasına sahiptir; birbirinden bağımsız çalıştırılabilir ve deploy edilebilir.
- **models/** — Veritabanı tablolarını temsil eden ORM (SQLAlchemy) sınıfları. “Veri neye benziyor, DB’de nasıl duruyor?” sorusuna cevap verir.
- **schemas/** — Pydantic modelleri; API’ye gelen/giden verinin (request/response) doğrulama ve şeklini tanımlar. models ile karıştırılmamalı: models veritabanı katmanı, schemas API katmanıdır.
- **services/ (her mikroservis içindeki alt klasör)** — İş mantığının (business logic) yaşadığı yer. Router’lar burayı çağırır, burada asıl işlem yapılır (örn. chunking_service.py metni parçalara böler).
- **routers/ (controllers)** — HTTP endpoint tanımları burada yapılır. Gelen isteği alır, ilgili service fonksiyonunu çağırır, sonucu schema formatında döner. İnce bir katman olmalı, iş mantığı burada değil services/ içinde olmalı.
- **shared/** — Tüm mikroservislerin ortak kullandığı kod. Örneğin JWT doğrulama mantığı 4 serviste de aynıysa, kopyala-yapıştır yerine burada tek bir yerde tutulur ve her serviste import edilir (Docker build sırasında bu klasör her servise kopyalanabilir ya da bir Python paketi olarak kurulabilir).
- **Dockerfile’lar** — Her mikroservisin kendi klasöründe kendi Dockerfile’ı bulunur; bu sayede her servis bağımsız olarak build edilip deploy edilebilir.
- **.env dosyaları** — Her serviste bir .env.example bulunur (repoya gerçek .env **asla** eklenmez, .gitignore’da tutulur). Gerçek .env dosyaları geliştirici kendi makinesinde oluşturur; Docker Compose çalıştırılırken env_file direktifiyle ilgili servise bağlanır. Ortak değişkenler (örn. JWT_SECRET) ana dizindeki .env dosyasından docker-compose.yml aracılığıyla tüm servislere aktarılabilir.

7. API Contract / Endpoint Planı

Auth Service

Method	Endpoint	Açıklama
POST	/auth/register	Şirket admini kaydı
POST	/auth/login	Giriş, JWT döner
GET	/auth/me	Giriş yapmış kullanıcı bilgisi

Örnek İstek — POST /auth/login

```
{
  "email": "admin@sirket.com",
  "password": "guvenli-sifre123"
}
```

Örnek Cevap

```
{
  "success": true,
  "data": {
    "access_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  }
}
```

```

    "token_type": "bearer",
    "user": {
      "id": "usr_123",
      "email": "admin@sirket.com",
      "role": "admin",
      "company_id": "cmp_001"
    }
  },
  "error": null
}

```

Document Service

Method	Endpoint	Açıklama
POST	/documents/upload	Doküman yükleme (multipart/form-data)
GET	/documents	Şirketin dokümanlarını listele
GET	/documents/{id}	Tek doküman detayı
DELETE	/documents/{id}	Doküman silme

Örnek Cevap — GET /documents

```

{
  "success": true,
  "data": [
    {
      "id": "doc_45",
      "filename": "hizmet_sozlesmesi_2025.pdf",
      "uploaded_by": "usr_123",
      "uploaded_at": "2026-06-01T10:15:00Z",
      "status": "processed"
    }
  ],
  "error": null
}

```

Ingestion Service (genelde iç servis, doğrudan frontend'e açık değil)

Method	Endpoint	Açıklama
POST	/ingestion/process	Document Service tarafından tetiklenir, doküman işlemeyi başlatır
GET	/ingestion/status/{document_id}	İşlem durumu sorgulama

Chat Service

Method	Endpoint	Açıklama
POST	/chat/ask	Kullanıcının sorusunu gönderir, cevap + kaynak döner
GET	/chat/history	Kullanıcının sohbet geçmişini getirir

Örnek İstek — POST /chat/ask

```
{
  "question": "Bu müşteriye daha önce nasıl teklif vermiştik?",
  "conversation_id": "conv_88"
}
```

Örnek Cevap

```
{
  "success": true,
  "data": {
    "answer": "Bu müşteriye 2025 yılında sunulan teklifte, 6 aylık danışmanlık paketi ve %15",
    "sources": [
      {
        "document_id": "doc_45",
        "document_name": "abc_musteri_teklif_2025.pdf",
        "page": 2,
        "excerpt_preview": "6 aylık danışmanlık paketi, %15 indirim..."
      }
    ],
    "conversation_id": "conv_88"
  },
  "error": null
}
```

Ortak Response Formatı

Tüm servisler aynı zarfı (envelope) kullanmalı — frontend'in her serviste farklı format ile uğraşmaması için:

```
{
  "success": true,
  "data": { },
  "error": null
}
```

Hata Response Formatı

```
{
  "success": false,
  "data": null,
  "error": {
    "code": "DOCUMENT_NOT_FOUND",
    "message": "Belirtilen doküman bulunamadı.",
    "details": null
  }
}
```

Yaygın hata kodları önerisi: UNAUTHORIZED, VALIDATION_ERROR, DOCUMENT_NOT_FOUND, INGESTION_FAILED, LLM_API_ERROR, INTERNAL_ERROR.

Servisler arası veri alışverişi

- **Document Service → Ingestion Service:** Dosya yüklendikten sonra document_id ve dosya yolu, HTTP POST ile /ingestion/process endpoint'ine gönderilir.

- **Chat Service → Ingestion Service'in oluşturduğu veri:** Chat Service, doğrudan vektör veritabanını (pgvector tablosu) okur; Ingestion Service'i çağırır. Bu nedenle vektör tablosu şeması, iki servisin de bildiği ortak bir shared şema olmalı.
 - **Tüm servisler → Auth Service:** Gelen her istekte JWT, shared/auth_middleware.py aracılığıyla doğrulanır; Auth Service'e her istekte ayrı bir çağrı yapmak yerine JWT içine company_id ve role bilgisi gömülür (stateless doğrulama).
-

8. Geliştirme Yol Haritası

Aşama 1 — Proje Setup (2-3 gün)

Çıktılar: - Repo yapısı yukarıdaki klasör ağacına göre oluşturulmuş. - docker-compose.yml iskeleti (servisler henüz boş "hello world" düzeyinde olabilir). - PostgreSQL container'ı ayakta, pgvector eklentisi kurulu. - .env.example dosyaları hazır. - Git branch stratejisi belirlenmiş (örn. main, develop, her özellik için feature/xxx).

Sorumluluk önerisi: Tüm ekip birlikte (mimariyi herkes anlamalı); repo sahibi/DevOps rolü bir kişi liderlik etsin.

Aşama 2 — Shared Modeller ve Ortak Altyapı (2 gün)

Çıktılar: - shared/auth_middleware.py, shared/response_format.py, shared/logger.py yazılmış. - Veritabanı şeması (ER diagram) çizilmiş: users, companies, documents, document_chunks, conversations, messages tabloları. - Alembic (migration aracı) kurulmuş.

Sorumluluk: 1 kişi shared modülleri, 1 kişi DB şema tasarımını yapabilir.

Aşama 3 — Backend Servisleri (5-6 gün)

Çıktılar: - Auth Service: register/login/JWT çalışıyor. - Document Service: dosya upload, listeleme, silme çalışıyor. - Ingestion Service: PDF/DOCX'ten metin çıkarma + chunking çalışıyor (henüz embedding yok). - Her servis kendi Swagger dokümantasyonuna sahip.

Sorumluluk: Auth + Document Service → 1 kişi; Ingestion Service → 1-2 kişi (parsing kütüphaneleri ile uğraşmak zaman alabilir).

Aşama 4 — AI Entegrasyonu (4-5 gün)

Çıktılar: - Embedding üretimi (Ingestion Service'e eklenir), vektörler pgvector'a kaydediliyor. - Chat Service: soru alma → vektör arama → LLM çağrısı → cevap üretme akışı uçtan uca çalışıyor. - Prompt şablonu hazırlanmış ("sadece verilen context'e dayan, uydurma" talimatı içeren). - Kaynak gösterimi (sources array'i) doğru şekilde dönüyor.

Sorumluluk: AI/LLM entegrasyonuna en yatkın 1-2 kişi bu aşamaya odaklansın; bu MVP'nin en kritik ve en riskli bölümüdür, erken başlanmalı.

Aşama 5 — Frontend (5-6 gün, Aşama 3-4 ile paralel yürütülebilir)

Çıktılar: - Login sayfası, Document upload/listeleme sayfası, Chat sayfası. - API entegrasyonları (authApi.ts, documentApi.ts, chatApi.ts). - Kaynak gösteren SourceCard bileşeni.

Sorumluluk: Frontend'e yatkın 1-2 kişi; backend API contract'ı (Bölüm 7) netleştigi anda paralel başlayabilirler (mock data ile).

Aşama 6 — Docker Compose Entegrasyonu (2-3 gün)

Çıktılar: - Tüm servisler docker-compose up ile tek komutla ayağa kalkıyor. - Nginx gateway servisleri doğru route ediyor. - Servisler arası network, environment variable'lar doğru çalışıyor.

Sorumluluk: DevOps/repo sahibi + gerektiğinde tüm ekip debug için destek versin.

Aşama 7 — Test ve Demo Hazırlığı (3-4 gün)

Çıktılar: - Uçtan uca senaryo test edilmiş: kayıt ol → doküman yükle → soru sor → doğru kaynaklı cevap al. - En az 3-4 örnek doküman ve gerçekçi demo senaryosu hazırlanmış (örn. gerçek bir sözleşme örneği, bir izin prosedürü metni). - README.md, kurulum talimatları, mimari diyagramı tamamlanmış. - Sunum/demo akışı prova edilmiş.

Sorumluluk: Tüm ekip; QA/test odaklı 1 kişi liderlik edebilir.

9. Karşılaşılabilecek Problemler ve Çözüm Önerileri

Problem	Neden Olur	Çözüm Önerisi
Mikroservis iletişim hataları (404, timeout)	Servis adresleri/portları yanlış, servis henüz ayağa kalkmamış	Docker Compose'da depends_on + health check kullan; servis isimlerini (container name) DNS gibi kullan, hardcoded IP kullanma
Docker build/network sorunları	Yanlış Dockerfile, port çakışması, network tanımsız	Her servisi önce tek başına docker build ile test et; docker-compose.yml'de ortak bir network tanımla
Environment variable problemleri	.env dosyası eksik/yanlış yüklenmiş, secret'lar farklı serviste farklı	Tek bir kaynak .env.example şablonu tut, docker-compose.yml'de env_file kullan, README'de hangi değişkenin zorunlu olduğunu listele
AI model/API hataları (rate limit, timeout, yanlış cevap)	LLM API kotası dolmuş, prompt kalitesiz, context çok uzun	Retry mekanizması ekle, prompt'u küçük test senaryolarıyla iyileştir, context uzunluğunu chunk boyutunu ayarlayarak sınırla
Embedding/vektör arama kalitesiz sonuç dönmesi	Chunk boyutu çok büyük/küçük, embedding modeli soruyla uyumsuz	Chunk boyutunu (örn. 500-800 karakter, %10-15 overlap) deneyerek ayarla; aynı embedding modelini hem doküman hem soru için kullan
Veritabanı şema uyuşmazlıkları	Servisler farklı zamanlarda farklı şema varsayımıyla geliştirilmiş	Migration aracı (Alembic) kullan, şema değişikliklerini PR review sürecine dahil et, ortak bir "veri sözlüğü" dokümanı tut
Takım içi branch/merge problemleri	Herkes main'e direkt push yapıyor, çakışan değişiklikler	feature/xxx branch stratejisi + PR review zorunluluğu; küçük, sık commit'ler; her mikroservis farklı klasörde olduğu için çakışma riski zaten azalır

Problem	Neden Olur	Çözüm Önerisi
Deployment sorunları (local’de çalışıyor, sunucuda çalışmıyor)	Ortam farklılıkları, eksik environment variable, port erişim izinleri	Local geliştirmeyi de Docker Compose ile yap (parity sağlanır), deployment öncesi checklist hazırla
Chunking/parsing hataları (bozuk PDF, taranmış görüntü PDF)	Bazı PDF’ler metin katmanı içermez (sadece görüntü)	MVP’de sadece metin tabanlı PDF/DOCX destekle, OCR desteğini kapsam dışı bırak; desteklenmeyen dosya için net hata mesajı göster
LLM’in “halüsinasyon” yapması (uydurma cevap)	Context yetersiz veya prompt zayıf	Prompt’a net kısıtlama ekle (“sadece verilen bilgiye dayan, bilmiyorsan bilmediğini söyle”); düşük similarity skorlu sonuçlarda “bu konuda dokümanlarda yeterli bilgi bulamadım” cevabı döndür

10. İlgi Çekici Ek Özellikler (MVP Sonrası, Zaman Kalırsa)

MVP tamamlandıktan sonra, zaman kalırsa veya jüri/demo etkisini artırmak için eklenebilecek, **karmaşıklık düşük ama etkisi yüksek** özellikler:

Güven Skoru / Belirsizlik Göstergesi

Chat cevabının yanında küçük bir “güven seviyesi” göstergesi (Yüksek/Orta/Düşük) gösterilebilir. Bu, vektör aramadaki similarity skoruna dayanır — eğer bulunan doküman parçaları soruyla düşük benzerlikteyse, kullanıcıya “bu cevap sınırlı bilgiyle üretildi, doğrulamayı öneririz” gibi bir uyarı gösterilir. Teknik olarak ekstra bir şey geliştirmeye gerek yok, zaten hesaplanan similarity skorunu frontend’e taşımak yeterli — ama kullanıcı güvenini ciddi şekilde artırır.

“Bu Cevap Yardımcı Oldu mu?” Geri Bildirimi

Her cevabın altına basit bir beğen/beğenme butonu eklemek. Bu veriler ileride: - Hangi dokümanların/cevapların zayıf olduğunu görmek için kullanılabilir. - “Sistem zamanla öğreniyor” hikayesinin ilk adımı olarak demo’da anlatılabilir (gerçek bir öğrenme mekanizması olmasa bile, veri toplama altyapısı kurulmuş olur).

Teknik olarak sadece messages tablosuna bir feedback alanı (up/down/null) eklemek yeterli.

Haftalık Özet / Sindirilebilir Bildirim

“Bu hafta en çok sorulan 3 konu şuydu” gibi basit bir özet — admin’e haftada bir e-posta ya da uygulama içi bildirim olarak gösterilebilir. Karmaşık bir analiz motoru gerektirmez; messages tablosundaki soruları haftalık gruplandırıp LLM’e “bu soruları 3 başlıkta özetle” diye sorup göstermek yeterlidir. Basit ama “akıllı” hissi veren bir özellik.

Doküman Güncellik Uyarısı

Yüklenen bir doküman belirli bir süre (örn. 6 ay, 1 yıl) güncellenmediyse, admin paneline “bu doküman eski olabilir, gözden geçirmek ister misin?” şeklinde bir uyarı düşürülebilir. Şirket doküman-

larının zamanla eskiyip yanlış bilgi vermesi riskini azaltır; teknik olarak sadece uploaded_at tarihine göre basit bir zamanlayıcı/kontrol gerektirir.

Slack/Teams Bot Entegrasyonu (Basit Versiyon)

Tam entegrasyon MVP dışı bırakılmıştı, ama zaman kalırsa **çok basit, tek yönlü bir Slack bot** (sadece /soru [metin] komutuyla Chat Service'e istek atıp cevabı Slack kanalına yazan) yarım günde kurulabilir ve demo'da çok etkileyici görünür — “çalışanlar zaten kullandıkları araçtan sora-biliyor” mesajını verir.

Neden Dashboard'ı Şimdilik Kapsam Dışı Bıraktık?

Dashboard (kullanım istatistikleri, en çok sorulan konular, admin analitiği) gerçekten değerli bir özellik, ama MVP'nin temel değer önerisini değiştirmiyor — sistemin “çalıştığını” göstermek için gerekli değil. Yukarıdaki listeden **güven skoru** ve **feedback butonu**, dashboard'a kıyasla çok daha az efor ile çok daha somut bir “akıllı sistem” hissi yaratır. Zaman kalırsa ve ekip isterse, dashboard eklemek için gereken veri zaten (messages, feedback tabloları) MVP'de toplanmış olacağından, ileride eklemek kolaylaşır.

Özet

Bu proje planı, bir bootcamp ekibinin 4-6 hafta içinde gerçekçi şekilde tamamlayabileceği, ama gerçek bir problemi çözen ve profesyonel görünen bir MVP tanımlar. Kritik nokta: **RAG pipeline'ının (Aşama 4) mümkün olduğunca erken başlaması** ve ekibin geri kalan zamanını arayüz/entegrasyon cilalamaya ayırabilmesidir. Kapsamı MVP sınırları içinde tutmak (e-posta entegrasyonu, çoklu tenant, analitik gibi özellikleri ileri aşamaya bırakmak) bootcamp'in başarıyla tamamlanmasının en önemli garantisidir.